

**ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



BLM4522 PROJE TESLİM RAPORU

Proje 5: Veritabanı Yükseltme ve Sürüm Yönetimi

Mehmet Tunahan DÜZYOL

22290109

10.06.2026

Video Bağlantısı: <https://youtu.be/ipgVD7Cw-qA?si=Klrav9SnOihM0lcB>

İçindekiler

1. Giriş.....	3
1.1 Projenin Amacı.....	3
1.2 Kapsam.....	3
1.3 Kullanılan Ortam ve Araçlar.....	3
2. Sürüm Kontrol Altyapısı.....	3
2.1 Şema Değişiklik Logu (SchemaChangeLog).....	4
2.2 Veritabanı Versiyon Tablosu (DatabaseVersion).....	4
3. DDL Trigger — Şema Değişikliklerini Yakala.....	4
3.1 DDL Trigger Nedir?.....	4
3.2 trg_SchemaChangeAudit Trigger'ı.....	4
3.3 Trigger Doğrulaması.....	5
4. Şema Değişiklikleri.....	5
4.1 V1.1.0 — CustomerFeedback Modülü.....	5
4.2 V1.2.0 — Kategori Özelliği.....	5
5. Test ve Doğrulama.....	6
5.1 Prosedür Testi.....	6
5.2 View Testi.....	6
5.3 DDL Log Doğrulaması.....	6
6. Geri Dönüş Planı.....	6
6.1 Rollback Yaklaşımı.....	6
6.2 V1.2.0 Geri Alımı.....	6
6.3 Versiyon Geçmişi.....	6
7. Bulgular ve Değerlendirme.....	7
7.1 Özet.....	7
7.2 Karşılaşılan Zorluklar.....	7
7.3 Sonuç.....	8
8. Kaynakça.....	8

1. Giriş

1.1 Projenin Amacı

Bu proje, Microsoft SQL Server 2025 ortamında AdventureWorks2025 örnek veritabanı üzerinde kapsamlı bir veritabanı sürüm yönetimi sistemi tasarlamak ve uygulamak amacıyla gerçekleştirilmiştir. Yazılım geliştirme süreçlerinde kaynak kod versiyonlaması ne kadar kritikse, veritabanı şema değişikliklerinin izlenmesi de en az o kadar önemlidir. Proje kapsamında DDL Trigger mekanizmasıyla şema değişiklikleri otomatik olarak kayıt altına alınmış, sürüm tablosu aracılığıyla her değişiklik belirli bir versiyon numarasına bağlanmış ve geri dönüş planları hazırlanarak test edilmiştir.

1.2 Kapsam

Proje beş ana başlık altında yapılandırılmıştır:

- Sürüm Kontrol Altyapısı: Şema değişiklik logu ve versiyon takip tablosunun oluşturulması
- DDL Trigger: CREATE, ALTER ve DROP olaylarının otomatik olarak yakalanması
- Şema Değişiklikleri: V1.1.0 ve V1.2.0 sürümlerinde gerçek değişikliklerin uygulanması
- Test ve Doğrulama: Oluşturulan nesnelerin işlevselliğinin test edilmesi
- Geri Dönüş Planı: V1.2.0 sürümünün geri alınması ve V1.1.1 olarak kaydedilmesi

1.3 Kullanılan Ortam ve Araçlar

Bileşen	Detay
Veritabanı Sistemi	Microsoft SQL Server 2025
Yönetim Arayüzü	SQL Server Management Studio (SSMS)
Test Veritabanı	AdventureWorks2025
Sürüm Yöntemi	DDL Trigger + T-SQL tabanlı versiyon tablosu

2. Sürüm Kontrol Altyapısı

Veritabanı sürüm yönetiminin temeli, şema üzerinde gerçekleştirilen her değişikliğin kim tarafından, ne zaman, hangi nesne üzerinde ve hangi T-SQL komutuyla yapıldığının kayıt altına alınmasına dayanır. Bu bilgiler olmaksızın bir veritabanının zaman içindeki evrimini takip etmek, sorunların kökenini tespit etmek ve gerektiğinde önceki duruma dönmek mümkün değildir.

2.1 Şema Değişiklik Logu (SchemaChangeLog)

dbo.SchemaChangeLog tablosu, veritabanı üzerinde gerçekleşen her DDL olayının kaydedildiği merkezi denetim tablosudur. Tabloda EventType, ObjectName, ObjectType, SchemaName, TSQLCommand, LoginName, HostName ve ChangeDate kolonları bulunmaktadır. Version kolonu ise ChangeID değerinden otomatik olarak türetilen hesaplanmış bir kolondur; her değişikliğe v1, v2, v3 gibi ardışık bir sürüm etiketi atanmaktadır. Bu yapı sayesinde tablo üzerinde manuel güncelleme yapmaya gerek kalmadan her yeni kayıt otomatik olarak versiyonlanmaktadır.

2.2 Veritabanı Versiyon Tablosu (DatabaseVersion)

dbo.DatabaseVersion tablosu, yazılım dünyasındaki migration tabloları ile aynı mantığa sahiptir. Her uygulanan değişiklik paketi bir versiyon numarasıyla (1.0.0, 1.1.0, 1.2.0 gibi), açıklamasıyla, uygulayan kullanıcıyla, uygulama tarihiyle ve en önemlisi geri dönüş betiğiyle birlikte bu tabloya kaydedilir. RollbackScript kolonu, ilgili sürümü geri almak için çalıştırılması gereken T-SQL kodunu saklamaktadır. Bu yaklaşım, acil durumlarda hangi komutların çalıştırılması gerektiğini arama ya da ezberlemek zorunda kalmaksızın doğrudan tablodan okuyarak uygulamayı mümkün kılmaktadır.

Başlangıç versiyonu olan 1.0.0, AdventureWorks2025 veritabanının projeye alındığı andaki taban durumunu temsil etmektedir ve geri dönüş betiği gerektirmemektedir.

3. DDL Trigger

3.1 DDL Trigger Nedir?

SQL Server'da trigger mekanizması yalnızca DML (INSERT, UPDATE, DELETE) olayları için değil, DDL (Data Definition Language) olayları için de kullanılabilir. DDL Trigger'lar; CREATE TABLE, ALTER TABLE, DROP TABLE, CREATE PROCEDURE gibi şema değişikliği komutlarını dinler ve bu olaylar gerçekleştiğinde otomatik olarak tetiklenir. Sunucu seviyesinde veya veritabanı seviyesinde tanımlanabilirler.

3.2 trg_SchemaChangeAudit Trigger'ı

Proje kapsamında trg_SchemaChangeAudit adında bir veritabanı seviyesi DDL Trigger oluşturulmuştur. Bu trigger; CREATE TABLE, ALTER TABLE, DROP TABLE, CREATE INDEX, DROP INDEX, CREATE PROCEDURE, ALTER PROCEDURE, DROP PROCEDURE, CREATE VIEW, ALTER VIEW, DROP VIEW, CREATE FUNCTION, ALTER FUNCTION ve DROP FUNCTION olaylarını dinlemektedir. Trigger içinde EVENTDATA() fonksiyonu kullanılmaktadır. Bu fonksiyon, tetiklenen DDL olayına ait tüm bilgileri XML formatında döndürür. XML içinden .value() metodu ile EventType, ObjectName, ObjectType, SchemaName ve TSQLCommand alanları ayrıştırılarak dbo.SchemaChangeLog tablosuna yazılmaktadır. HOST_NAME() fonksiyonu ise değişikliğin hangi makineden yapıldığını kaydeder.

3.3 Trigger Doğrulaması

Trigger oluşturulduktan sonra sys.triggers sistem görünümü sorgulanmış ve trg_SchemaChangeAudit trigger'ının is_disabled = 0 yani aktif durumda olduğu doğrulanmıştır. parent_class_desc değerinin DATABASE olması, trigger'ın sunucu seviyesinde değil veritabanı seviyesinde tanımlandığını göstermektedir.

4. Şema Değişiklikleri

4.1 V1.1.0 — CustomerFeedback Modülü

İlk sürüm değişikliği kapsamında beş ayrı DDL işlemi gerçekleştirilmiştir. Bu işlemlerin her biri trg_SchemaChangeAudit trigger'ı tarafından otomatik olarak yakalanmış ve dbo.SchemaChangeLog tablosuna kaydedilmiştir.

Değişiklik 1 — Yeni Tablo: dbo.CustomerFeedback tablosu oluşturulmuştur. Tablo; FeedbackID (birincil anahtar, otomatik artan), CustomerID, FeedbackText, Rating (1—5 arası CHECK kısıtı) ve SubmittedDate kolonlarından oluşmaktadır.

Değişiklik 2 — Kolon Ekleme: Tabloya IsProcessed adında BIT tipinde, varsayılan değeri 0 olan yeni bir kolon eklenmiştir. Bu değişiklik, ALTER TABLE komutuyla mevcut yapıya dokunulmaksızın gerçekleştirilmiştir.

Değişiklik 3 — Index Oluşturma: CustomerID kolonu üzerinde Rating ve SubmittedDate kolonlarını kapsayan bir nonclustered index oluşturulmuştur. Bu index, müşteri bazlı geri bildirim sorgularının performansını artırmaktadır.

Değişiklik 4 — View Oluşturma: dbo.vw_TopRatedFeedback adında bir view oluşturulmuştur. Bu view; CustomerFeedback, Sales.Customer ve Person.Person tablolarını birleştirerek puanı 4 ve üzerinde olan geri bildirimleri müşteri adıyla birlikte sunmaktadır.

Değişiklik 5 — Stored Procedure: dbo.usp_AddFeedback prosedürü oluşturulmuştur. Prosedür; CustomerID, FeedbackText ve Rating parametrelerini alarak CustomerFeedback tablosuna yeni bir kayıt eklemekte ve oluşturulan kaydın ID'sini döndürmektedir.

Bu değişikliklerin tamamı V1.1.0 olarak dbo.DatabaseVersion tablosuna kaydedilmiş; geri dönüş betiği de aynı kayıta saklanmıştır.

4.2 V1.2.0 — Kategori Özelliği

İkinci sürüm değişikliği kapsamında mevcut yapıya iki ekleme yapılmıştır.

Değişiklik 6 — Kolon Ekleme: dbo.CustomerFeedback tablosuna Category adında NVARCHAR(50) tipinde, varsayılan değeri 'Genel' olan yeni bir kolon eklenmiştir.

Değişiklik 7 — Prosedür Güncelleme: dbo.usp_AddFeedback prosedürü yeni @Category parametresini içerecek şekilde güncellenmiştir. Ayrıca Rating aralık denetimi için RAISERROR tabanlı girdi doğrulama eklenmiştir. Bu güncelleme geriye dönük uyumluluğu korumaktadır: @Category parametresinin varsayılan değeri 'Genel' olarak tanımlandığından mevcut çağrılar parametre geçmeksizin çalışmaya devam etmektedir.

Bu değişiklikler V1.2.0 olarak kaydedilmiş; geri dönüş betiğinde Category kolonunun kaldırılması ve prosedürün önceki haline döndürülmesi için gereken T-SQL kodları saklanmıştır.

5. Test ve Doğrulama

5.1 Prosedür Testi

dbo.usp_AddFeedback prosedürü hem V1.1.0 hem de V1.2.0 yapısıyla test edilmiştir. İlk testte CustomerID 1 için beş yıldızlı bir geri bildirim 'Ürün Kalitesi' kategorisiyle eklenmiştir. İkinci testte CustomerID 2 için üç yıldızlı bir geri bildirim 'Teslimat' kategorisiyle eklenmiştir. Her iki testte de prosedür başarıyla çalışmış ve yeni oluşturulan kaydın ID'si döndürülmüştür.

5.2 View Testi

dbo.vw_TopRatedFeedback view'ı sorgulanmış ve yalnızca Rating değeri 4 ve üzerinde olan kayıtların görüldüğü doğrulanmıştır. View; müşteri adını, geri bildirim metnini ve puanı doğru şekilde birleştirerek sunmaktadır.

5.3 DDL Log Doğrulaması

dbo.SchemaChangeLog tablosu sorgulanmış ve V1.1.0 ile V1.2.0 kapsamında gerçekleştirilen tüm DDL işlemlerinin — CREATE TABLE, ALTER TABLE, CREATE INDEX, CREATE VIEW, CREATE PROCEDURE, ALTER PROCEDURE — eksiksiz olarak loglandığı doğrulanmıştır. Her kayıta EventType, nesne adı, şema adı, uygulayan kullanıcı ve çalıştırılan T-SQL metni yer almaktadır.

6. Geri Dönüş Planı

6.1 Rollback Yaklaşımı

Geri dönüş planı, dbo.DatabaseVersion tablosundaki RollbackScript kolonunda saklanan T-SQL betiklerinin çalıştırılmasına dayanmaktadır. Bu yaklaşımın avantajı, acil bir durumda DBA'nın hangi komutları çalıştıracağını araştırmasına gerek kalmamasıdır; ilgili versiyon satırındaki RollbackScript değeri okunarak doğrudan uygulanabilir.

6.2 V1.2.0 Geri Alımı

V1.2.0 sürümü geri alınırken iki işlem gerçekleştirilmiştir. İlk olarak dbo.CustomerFeedback tablosundan Category kolonu kaldırılmıştır. Ardından dbo.usp_AddFeedback prosedürü @Category parametresi olmayan önceki haline döndürülmüştür. Bu geri alım işlemi V1.1.1 olarak dbo.DatabaseVersion tablosuna kaydedilmiştir.

6.3 Versiyon Geçmişi

Tüm versiyon geçmişi dbo.DatabaseVersion tablosundan sorgulanabilmektedir.

Versiyon	Açıklama	Tarih
1.0.0	Başlangıç versiyonu — taban çizgisi	Proje başlangıcı
1.1.0	CustomerFeedback tablosu, view ve stored procedure eklendi	V1.1.0 uygulaması
1.2.0	Category kolonu ve kategori parametresi eklendi	V1.2.0 uygulaması
1.1.1	V1.2.0 geri alındı	Rollback uygulaması

7. Bulgular ve Değerlendirme

7.1 Özet

Tespit / Uygulama	Kullanılan Yöntem	Sonuç
Şema değişiklik logu oluşturuldu	SchemaChangeLog tablosu	Her DDL olayı otomatik kaydedildi
DDL Trigger kuruldu	trg_SchemaChangeAudit	14 farklı DDL olay türü dinleniyor
V1.1.0 uygulandı	5 DDL işlemi	Tüm değişiklikler loglandı
V1.2.0 uygulandı	2 DDL işlemi	Geri dönüş betiğiyle birlikte kaydedildi
V1.2.0 geri alındı	RollbackScript çalıştırıldı	Veritabanı V1.1.0 yapısına döndü
Versiyon geçmişi raporlandı	DatabaseVersion tablosu	4 versiyon kaydı oluşturuldu

7.2 Karşılaşılan Zorluklar

Proje sürecinde dikkat edilmesi gereken birkaç teknik husus tespit edilmiştir. DDL Trigger içinde EVENTDATA() fonksiyonunun döndürdüğü XML'den değer ayrıştırırken XQuery .value() metodunun doğru namespace ve yol ifadeleriyle kullanılması gerekmektedir; yanlış yol ifadesi NULL değer dönmesine yol açmaktadır. Bunun yanı sıra DDL Trigger'ın DISABLE TRIGGER komutuyla geçici olarak devre dışı bırakılabileceği; ancak bu sürede yapılan değişikliklerin loglanmayacağı unutulmamalıdır. Son olarak RollbackScript kolonundaki T-SQL metinlerinin tek tırnak kaçış karakterleriyle doğru şekilde saklanması, geri dönüş sırasında söz dizimi hatalarını önlemek açısından kritik öneme sahiptir.

7.3 Sonuç

Bu proje kapsamında Microsoft SQL Server 2025 üzerinde AdventureWorks2025 veritabanı kullanılarak tam işlevli bir veritabanı sürüm yönetimi sistemi tasarlanmış ve uygulanmıştır. DDL Trigger mekanizmasıyla şema üzerindeki her değişiklik kim tarafından, ne zaman ve hangi komutla yapıldığına dair bilgilerle birlikte otomatik olarak kayıt altına alınmaktadır. Versiyon tablosu sayesinde her değişiklik paketi numaralandırılmış ve geri dönüş betiğiyle ilişkilendirilmiştir. V1.2.0 sürümünün başarıyla geri alınması, tasarlanan sistemin üretim ortamı gereksinimlerini karşıladığını doğrulamaktadır.

8. Kaynakça

- Microsoft Docs — DDL Triggers:
<https://learn.microsoft.com/sql/relational-databases/triggers/ddl-triggers>
- Microsoft Docs — EVENTDATA Function:
<https://learn.microsoft.com/sql/t-sql/functions/eventdata-transact-sql>
- Microsoft Docs — CREATE TRIGGER:
<https://learn.microsoft.com/sql/t-sql/statements/create-trigger-transact-sql>
- Microsoft Docs — sys.triggers:
<https://learn.microsoft.com/sql/relational-databases/system-catalog-views/sys-triggers-transact-sql>
- AdventureWorks Sample Database:
<https://github.com/microsoft/sql-server-samples>